

Expressible but Not Learnable? MDL Local-Rule Induction and the ARC-AGI-1 → ARC-AGI-2 Gap

ARC Prize 2026 — Paper Track submission

Abstract

We present a CPU-only ARC solver built around one principle: **choose the shortest consistent explanation, and say when there is none**. The solver searches over grid-level transforms and then induces *local rules*. A local rule is a lookup table from a small tuple of per-cell features (own colour, neighbours, rays, symmetry partners, object statistics, grid-level context) to an output label. Candidate rules are scored by description length (DL). Predictions from all consistent rules are pooled by Bayesian model averaging with weights $\propto 2^{-DL}$. Three mechanisms make the rule class generalise beyond memorisation:

1. **target re-encodings** (*absolute, keep-unless-changed and copy-from-feature*);
2. **monotone interval interpolation**, so rules keyed on ordinal features stay defined for values never seen in training;
3. a **compression gate** that rejects rules which do not compress the observed changes.

We also test two ideas that did *not* help, and report them as negative results: meta-learning the DL prior over features, and averaging predictions over all consistent rules.

Our main contribution is diagnostic. We separate *expressibility* (a compact rule exists, found by an oracle that also sees the test outputs) from *learnability* (the rule is recovered from the training pairs alone). On the 1,000 ARC-AGI-2 training tasks, compact local rules express **187** tasks (18.7%), and the solver learns **126 of those (67%)** from the demonstrations alone. On the 120 public evaluation tasks, the same rule class expresses **only 2 (1.7%)**, and neither is learned. The ARC-AGI-2 evaluation set is therefore not merely harder to *learn*: it is almost entirely outside the *expressible* region of per-cell rules, even with a rich feature bank and grid-level context. We use this to argue, quantitatively, where program-synthesis research effort should go.

1. Introduction

ARC-AGI asks a solver to infer a grid transformation from two to five demonstration pairs. Two families dominate the literature. Neural methods use test-time-trained LLMs or recursive networks. Symbolic methods search a DSL. Both families improve on ARC-AGI-1 and struggle on ARC-AGI-2. The reason is usually described qualitatively: ARC-AGI-2 tasks are "more compositional".

We make this quantitative for one well-defined, interpretable hypothesis class: **local rules**. A local rule states that each output cell is a function of a few features of the corresponding input cell (after an optional global transform). The class is small enough to enumerate exhaustively. It is rich enough to cover recolouring by object size, symmetry completion, hole filling, line extension between markers, parity patterns, and conditioning on a "key" object elsewhere in the grid. Because the class can be enumerated, we can answer two separate questions for every task:

- **Expressibility**. Does *any* compact rule reproduce the train and test outputs? We answer this with an oracle that is given the test outputs.
- **Learnability**. Does MDL selection, from the train pairs alone, pick a rule that is right on the test?

The difference between the two is the *induction gap*. Tasks outside the expressible region measure the *representation gap*. Keeping the two apart tells us whether to improve priors (the induction gap) or the hypothesis class (the representation gap).

2. Method

2.1 Transform stage

A library of about 110 parameter-free grid transforms covers the following:

- rotations and reflections;
- crops to the non-background bounding box or to a selected object (largest, smallest, unique colour, unique shape, densest, most colours);
- integer up- and down-scaling;
- tilings and mirror tilings;
- a self-Kronecker product;
- separator-aware splitting into panels, combined by count, bit-code, AND, OR, XOR, first/last overlay, or selection of the most/least/unique panel;
- per-cell summaries of separator grids;
- de-duplication of rows and columns.

A transform is *exact* if it maps every train input to its output. Otherwise, if it produces output-shaped grids, it is kept as the substrate for rule induction. Depth-2 compositions are checked for exactness only. The six substrates closest to the targets (by pixel accuracy) are passed to the rule stage.

2.2 Feature bank

For every cell we compute about 70 integer features:

- **local**: own colour, the 8 neighbours, first non-background colour along each of 8 rays, horizontal/vertical "between two same-coloured cells", 4- and 8-neighbour counts, same-colour neighbours;
- **geometric**: row and column indices from each edge, index mod 2 and 3, border and diagonal membership, mirror partners (horizontal, vertical, both, transpose, anti-transpose), dominant colour of the row and of the column;
- **object**: for single-colour 4-connected objects and multicolour 8-connected objects: size, size rank from the top and from the bottom, border contact, number of colours, minority ("marker") colour, enclosure;
- **grid context**: number of objects, rarest and commonest colour, colour and shape hash of the smallest and largest object;
- **frequency**: frequency rank of the own colour.

2.3 Rule induction under MDL

For a feature subset S and a target encoding E , the rule is the table $\text{key}_S(\text{cell}) \rightarrow \text{label}_E(\text{cell})$. It is *consistent* if no key maps to two labels across all training cells. The target encodings are:

- **abs**: the output colour;
- **keep**: KEEP if the cell is unchanged, else its new colour;
- **copy_f**: KEEP if unchanged, else COPY, meaning "take the colour of feature f ". This is offered only when f explains every changed cell.

We enumerate all singletons and pairs exhaustively. We extend the 24 least-impure pairs to triples (beam search). The description length of a rule is

$$DL = |table| + \sum_{f \in S} cost(f) + cost(E) + 4 \cdot \#unseen + interp_cost.$$

Here $\#unseen$ counts test keys that the table does not cover (they default to KEEP). $interp_cost$ is charged per key resolved by interval interpolation (§2.4). Rules with more than 40 table entries fail the **compression gate**. They are kept only as a last resort.

2.4 Monotone interval interpolation

Object sizes, counts and ranks in the test grid are often values never seen in training. For an unseen test key, we look for seen keys that match it on all positions except one ordinal feature. If the nearest seen values below and above agree on the label, the key inherits that label (cost 1). One-sided extrapolation costs 2. Conflicts leave the key unresolved.

2.5 Bayesian model averaging

Every consistent rule, on every substrate, votes for its predicted test grids with log-weight $-DL/2 - cost(transform)$. Identical predictions pool their evidence with log-sum-exp. The two highest-scoring distinct predictions are submitted as `attempt_1` and `attempt_2`.

2.6 Meta-learned feature prior

The uniform $cost(f) = 1.5$ is replaced by $cost(f) = -\log_2 p(f) / \log_2 10$, measured in table-entry units. $p(f)$ is the Laplace-smoothed frequency of f in the minimal-DL oracle rules of held-in tasks. We report 2-fold cross-validation on the training set, so the prior is never estimated from the tasks it is evaluated on.

3. Experiments

All numbers come from `run_experiments.sh` on 4 CPU cores. The time limits are 20 s per task on training and 30 s on evaluation. The full training run takes about 4 minutes. A task's score is the fraction of its test outputs matched exactly by one of the two attempts.

3.1 Accuracy

System	ARC-AGI-2 training (1,000)	ARC-AGI-2 public eval (120)
Transforms + local rules (full)	135.5 (13.6%)	0 (0.0%)
... + learned feature prior	135.5 (13.6%, 2-fold CV)	0 (0.0%)

The ARC-AGI-2 training set contains most of ARC-AGI-1, and the solver solves 13.6% of it. It solves nothing on the public evaluation set. Precision on training correlates strongly with rule size. Rules with ≤ 5 table entries are right on 40.5 of the 47 tasks where they are chosen. Rules with ≤ 20 entries are right on 72 of 127. Rules with > 40 entries are right on only 5 of 233, which is what motivates the compression gate.

3.2 Expressibility vs. learnability (main result)

For each task we run the same machinery with the **test outputs included as training pairs** (the oracle) and ask whether a rule with ≤ 40 table entries, or an exact transform, reproduces every pair.

	Training (1,000)	Public eval (120)
Output shape reachable by some transform	817 (81.7%)	89 (74.2%)
Expressible (exact transform or compact rule)	187 (18.7%)	2 (1.7%)
... of which exact transforms	32	0
Solved from train pairs alone	137	0
Learned expressible	126 / 187 (67%)	0 / 2

Three observations:

1. **On ARC-AGI-2 evaluation, the representation gap dominates.** 89 of 120 tasks have output grids of the right *shape* after some transform. Yet per-cell rules over ~ 70 features (local, geometric, object-level and grid-context) express only two of them. Even these two need 24–32 table entries, close to the gate: they are near-memorisations, not concise laws. The ARC-AGI-1-style mechanisms (recolour by property, fill by enclosure, symmetry completion, panel overlays) are essentially absent from the new evaluation set.
2. **On ARC-AGI-1-style tasks, the induction gap is moderate.** Given an expressible task, MDL selection from 2–5 demonstrations picks a correct rule 67% of the time. Inspecting the failures shows two kinds. In the first, the oracle's rule is *coincidental*: object size happens to separate the test classes, while the true cause is the shape of a key object. In the second, the test contains feature values outside the training range. The first kind motivated the grid-context features, the second the interval interpolation.
3. **Learnability improvements do not transfer.** Every component that raised training accuracy left evaluation accuracy at exactly zero, because it acts inside a region that the evaluation tasks do not occupy.

3.3 Ablations (training set, 1,000 tasks)

Removed component	Score	Δ
none (full)	135.5	—
keep target encoding	122.0	−13.5
copy - from - feature target encoding	128.5	−7.0
grid-context features	132.0	−3.5
interval interpolation	133.5	−2.0
compression gate	135.5	0.0
Bayesian model averaging (keep only the min-DL rule)	136.0	+0.5

The **target encodings are the most important design choice**. Letting a rule output "unchanged" or "copy the colour of feature f " makes most tables small and colour-invariant. That is exactly the kind of generalisation the demonstrations cannot show directly. The compression gate does not change accuracy by design, since it only re-ranks candidates; its value is calibration. Model averaging brings no benefit: identical predictions from many near-duplicate rules (e.g. `c` vs. `col_rank`) inflate their total weight without adding independent evidence.

3.4 Learned feature prior (negative result)

Replacing the uniform per-feature cost with $-\log p(f)$, estimated from the oracle rules of the other fold, leaves the score *exactly* unchanged on both folds: $67.5 \rightarrow 67.5$ and $68.0 \rightarrow 68.0$. The reason is structural. Rule DL is dominated by the table size $|table|$. Feature costs differ by less than one table entry (0.8 for `c` up to 2.5 for rare

features), so they only break ties between rules of equal table size. Such ties almost always predict the same test output.

4. Discussion

Where should effort go? Our measurements give a concrete answer for symbolic ARC solvers. Better priors, averaging and interpolation over a *per-cell* hypothesis class buy a few points on ARC-AGI-1-style tasks and nothing on ARC-AGI-2. The ARC-AGI-2 evaluation tasks sit outside the class: 87 of 89 shape-reachable tasks cannot be written as a compact function of any cell-local or grid-context feature set we tried. To make progress, a solver must change the *unit* of the rule. It needs rules over objects and their relations (which object goes where, matched by shape up to symmetry), rules that apply other rules conditionally, and rules whose output geometry is computed rather than copied.

A diagnostic, not only a solver. The expressible/learnable split is cheap to compute: about 4 minutes on a CPU for 1,000 tasks. It can be run on any rule or program class for which an oracle fit exists. We suggest that ARC method papers report both numbers, because they separate two claims that are usually conflated: "my representation can describe these tasks" and "my inference procedure finds the description from few examples". For example, an LLM-based program synthesiser can be evaluated by (a) whether a program exists in its sampling distribution that fits train+test, and (b) whether it is selected from train alone.

Honest scope. A 13.6% training score is far below neural test-time-training systems. The contribution is the measurement, together with a fast, fully interpretable baseline whose every prediction comes with a human-readable rule (for example `keep-unless-changed on [enclosed] (2 entries)` for "fill enclosed regions").

5. Limitations

- The rule class is per-cell. It cannot express counting-to-output-size, object re-arrangement, or multi-step programs unless one of the substrate transforms already does the non-local work.
- The shape hashes in the context features are local to a grid. Matching shapes *across* demonstrations works by hash equality, which is brittle to rotation.
- Accuracy on ARC-AGI-2 is low in absolute terms. We make no claim that this approach is competitive with test-time-trained LLMs. Our claim is about *where* the difficulty lies.

6. Reproducibility

All code is open source (Apache-2.0) in this repository. The whole pipeline runs on four CPU cores in under ten minutes for the 1,000 training tasks.

- `evaluate.py` scores any task directory.
- `analysis_oracle.py` produces the expressibility/learnability numbers.
- `learn_prior.py` produces the feature prior.
- `kaggle/arc_mdl_rules.ipynb` is the offline Kaggle submission (no internet, no GPU, no pretrained weights).